The Silent Threat: Inconsistent ML Environments

By TEAM 6 : Sankalp Khamesra , Samruddhi D , Sasank Sekhar Panda and Shanavi Narayan

Machine learning models, once deployed, are often assumed to perform consistently. However, subtle differences in their operating environments can lead to significant and unexpected behavioral changes. This is particularly critical in sensitive applications like MedInsure's claim-risk scoring, where accuracy directly impacts financial decisions and regulatory compliance.

Python Version Mismatch

A shift from Python 3.8 (training) to 3.10 (staging) can introduce discrepancies in random number generation, hashing, and object serialization. This can cause models to behave differently, leading to approval score mismatches.

ML Library Version Mismatch

Variations in libraries like scikit-learn, XGBoost, or TensorFlow can alter underlying algorithm implementations, default hyperparameters, and data preprocessing pipelines. This results in inconsistent claim-risk scoring and unreliable pre-production decisions.

These environmental inconsistencies can lead to significant deviations between development and production, undermining model reliability and trust.

Ensuring Environment Consistency

To mitigate the risks of environmental inconsistencies, MedInsure must implement robust production-readiness steps. These solutions focus on standardizing environments, meticulously capturing dependencies, and aligning data pipelines.

Standardize Environments End-to-End

Utilize Docker containers to freeze the exact Python version (e.g., 3.8) and all ML library versions used during training. Promote the same container across development, staging, and production to eliminate environmental drift.

Capture & Version Control All Dependencies

Generate and version-control `requirements.txt` or `environment.yaml` files using tools like `pip-freeze` or `Poetry.lock`. Save the environment hash in a model registry (e.g., MLflow) for complete reproducibility.

Align Training & Deployment Data

Ensure preprocessing logic is identical across training, staging, and production ETL pipelines. Analyze monthly claim volumes, hospital type contributions, and diagnosis code shifts to prevent discrepancies caused by ETL updates or seasonal patterns.

Optimizing Performance and Stability with Caching & Scaling

Beyond environmental consistency, MedInsure faces performance challenges during peak loads and needs to adapt to seasonal claim patterns. Strategic caching and temporal scaling are crucial for maintaining API responsiveness and model stability.

Claim-Aware Caching

During load testing, the API experienced timeouts at 5,000 claims/min. Implementing a caching layer for frequently occurring, low-risk claims can dramatically reduce model calls and avoid performance bottlenecks.

- Cache flu treatment claims (cost 3k–5k).
- Cache minor fracture claims (10k–15k).
- Cache standard diagnostic tests (1k–3k).

These categories represent over 40% of low-risk submissions, significantly reducing the need for ML inference and minimizing environment-related inconsistencies.

Seasonal / Temporal Environment Scaling

MedInsure observes predictable claim spikes during monsoon (dengue, viral fever), winter (respiratory claims), and post-holiday periods (minor accidents). Proactive compute scaling before these known spikes is essential.

- Utilize the last 5 years of claim data patterns to predict and prepare for surges.
- Prevents API timeouts for the mobile app during peak usage.
- Ensures model stability under heavy load.
- Aligns resource allocation with real-world medical claim patterns.

Risk Management: Friction Scores & Segmentation

MedInsure can further enhance its production readiness by introducing innovative risk assessment tools. A "Friction Score" quantifies environmental drift, while "Hospital-Risk Segmentation" validates model behavior across diverse claim sources.

Environment "Friction Score"

Before deployment, compute a numerical score quantifying the deviation between staging and training environments. This score considers Python, library, OS, and CPU/GPU kernel differences, as well as preprocessing code diffs. This unique approach provides a concrete risk metric for environment drift.

Hospital-Risk Segmentation

Given the disappearance of `hospital_risk_score` from the ETL, segment testing by hospital type (urban private, Tier-2 public, rural clinics). This helps identify and address mismatches, particularly in rural clinics (diagnosis code changes) and private hospitals (cost range patterns), ensuring fair approval decisions.

Dependency Insurance: Future-Proofing Model Stability

With Python 3.8 nearing end-of-life, MedInsure faces the risk of future updates breaking its model. Implementing "Dependency Insurance" through automated checks ensures long-term environment stability and proactive migration strategies.

An automated tool should continuously monitor the status of critical dependencies, checking for:

- **Deprecation Warnings:** Is scikit-learn 0.24 nearing deprecation?
- **Breaking Changes:** Will pandas 1.2 introduce breaking changes in future OS updates?
- **Numerical Precision Issues:** Are there known numerical precision warnings in NumPy 1.19?

If the risk of incompatibility or instability is high, the system should automatically trigger a retraining process or a planned migration to newer, stable versions. This proactive approach prevents unexpected model failures and ensures continuous operational integrity.

Python 3.8's end-of-life poses a significant threat. Dependency Insurance ensures the model remains robust against evolving software ecosystems.

Achieving Reproducibility: Data Versioning

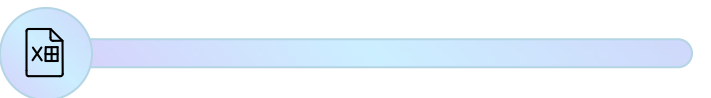
Full model reproducibility is paramount for MedInsure, especially given past issues with ETL changes and regulatory scrutiny. Versioning training data is the first critical step to ensure that model decisions can always be traced back to their exact data origins.

Historical Claims Dataset Snapshots

Store the exact 5-year dataset used for training, including claim IDs, hospital risk scores, diagnosis code distributions, and rare surgical procedure records. This immutable snapshot allows MedInsure to reproduce the precise data context for any past claim decision.

ETL Pipeline Versioning

Version control the ETL pipeline itself. This enables MedInsure to identify which ETL version produced a specific training dataset, when features like "hospital_risk_score" were removed, and when schema drift occurred. This transparency is vital for auditing and troubleshooting.



Immutable Claim-Data Snapshots

Create immutable snapshots, such as "Claims_Training_Data_v1.0_2018_to_2023.parquet". If a regulator questions a 2021 gallbladder surgery claim approval, MedInsure can instantly reproduce the exact 2021 data used during training, ensuring full accountability.

Achieving Reproducibility: Environment Metadata & Hyperparameters

Beyond data, the exact environment and hyperparameters used during model training are crucial for reproducibility. MedInsure must meticulously store this metadata to explain any discrepancies between development and production.

Storing Environment Metadata

To explain why staging scored a claim differently than development, MedInsure must store:

- **Software Versions:** Python (3.8.xx), NumPy, Pandas, scikit-learn.
- **OS Details:** Ubuntu version.
- **Serialization Format:** Pickle/joblib version.
- **Environment Hash:** A cryptographic hash of the entire training environment.
- **Docker Image ID:** For the specific training run.
- **Feature Engineering Code:** Exact code for `hospital_risk_score`, `cost_bucket`, etc.

Tracking Hyperparameters

Auditors frequently request hyperparameters, making their meticulous tracking essential. MedInsure must store:

- **Model Random Seed:** Critical for rare procedure handling.
- **Model Architecture:** Tree depth or neural network specifics.
- **Classification Thresholds:** For "low-risk" classification.
- **Feature Weighting:** Especially for fraud features.
- **Preprocessing Parameters:** Scaling ranges from historical data.

This is particularly important for features tied to rare surgical procedures, which have been flagged by auditors.

Regulatory Playback & Rare Procedure Registry

MedInsure needs specialized systems to address unique challenges like regulatory inquiries and poor model performance on rare cases. Implementing a "Regulatory Playback Mode" and a "Rare Procedure Registry" ensures comprehensive reproducibility and accountability.

1

Regulatory Playback Mode

Create a system to replay the exact training environment, historical claim, preprocessing logic, and hyperparameters for any automatically approved claim under ₹25,000. This allows regulators to "replay" the model's decision, restoring the exact Python 3.8 environment used at the time, especially for escalated rejected claims or fraud cases.

2

Rare Procedure Registry

Since the model performed poorly on rare surgical procedures, establish a versioned index of all rare procedure claims used during training. This includes a timestamped log for when such cases enter production and a data snapshot for each rare-code claim. This ensures exact replay and retraining of how rare procedures were historically treated.

Feature Removal Alert System & Seasonality Imprints

MedInsure's reproducibility strategy must also account for dynamic data environments and seasonal claim patterns. Implementing a "Feature Death Certificate" system and "Claim Seasonality Imprints" provides robust mechanisms for tracking changes and ensuring model integrity.

Feature Removal Alert System

If an important feature like *hospital_risk_score* is dropped from ETL, (which caused 22% claim rejections), the system logs the removal, identifies impacted models, highlights schema changes, and automatically triggers retraining or shadow testing. This prevents unnoticed feature loss from causing production failures.

Claim Seasonality Imprint

Given that low-risk claim volumes spike in certain months (e.g., flu season), create versioned seasonality profiles. Store month-wise low-risk claim frequency, fraud flag occurrence, and hospital type distribution for each training dataset version. This allows auditors to check if changes in seasonal claim patterns alter decision patterns when the model is retrained.

Adversarial Ledgers & Environment DNA

To combat sophisticated attacks and ensure absolute traceability, MedInsure needs to implement advanced reproducibility measures. An "Adversarial Diagnosis Code Ledger" and "Environment DNA Fingerprinting" provide the highest level of security and accountability.

Adversarial Diagnosis Code Ledger

Since attackers manipulated diagnosis codes, version every diagnosis code mapping table, fraud-flag rules, and manipulated model inputs. This allows MedInsure to replicate exact attack scenarios, crucial for understanding and defending against future threats.

Environment DNA Fingerprinting

Generate a cryptographic fingerprint for Python 3.8, NumPy, scikit-learn, and custom fraud-flag feature engineering logic. Attach this unique fingerprint to EVERY model prediction. If a claim is escalated, MedInsure knows exactly which environment produced the decision, ensuring irrefutable traceability.